# Praktikum 14

**Realtime Local Notification Antar Perangkat**

| | | |
|---|---|---|
| Topik | : | Flutter Local Notification & Realtime App (WebSocket) |
| Backend | : | Node.js + Socket.IO |
| Client | : | Flutter UI + flutter_local_notifications + socket_io_client |

---

# 1. Pendahuluan

Pada praktikum ini, kita akan membangun aplikasi Flutter sederhana yang dapat menampilkan **notifikasi lokal** secara **realtime** antar beberapa perangkat khususnya Android.

Tujuan utama praktikum:

1. Mengidentifikasi cara kerja **realtime flutter local notification** menggunakan Socket.IO.
2. Menjelaskan bagaimana **event dari server** digunakan untuk memicu **local notification** di Flutter.
3. Menguji aplikasi pada **beberapa perangkat sekaligus** dan menganalisis perilaku notifikasi.

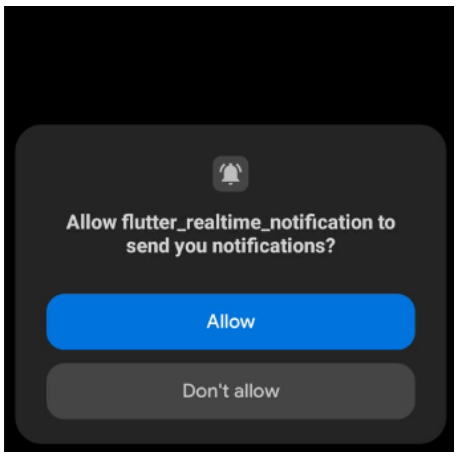
Skenario utama:

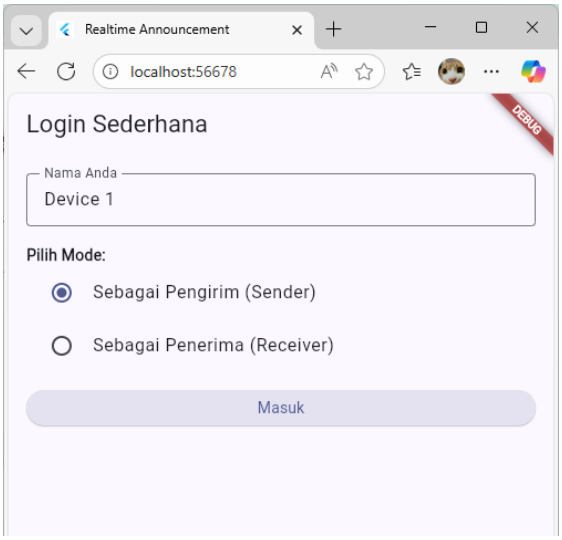
- Terdapat beberapa perangkat: A, B, dan C.
- Ketika **A mengirim pengumuman**, maka **hanya B dan C** yang menerima notifikasi lokal.
- Ketika **B mengirim pengumuman**, maka **hanya A dan C** yang menerima notifikasi.
- Pengirim **tidak** menerima notifikasi dari pesan yang ia kirim sendiri.

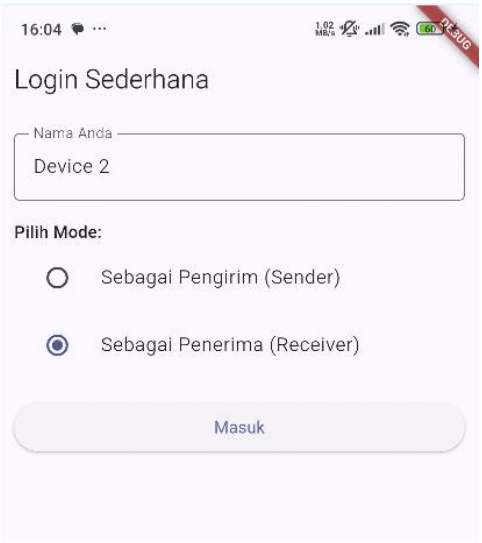
Studi kasus yang digunakan:

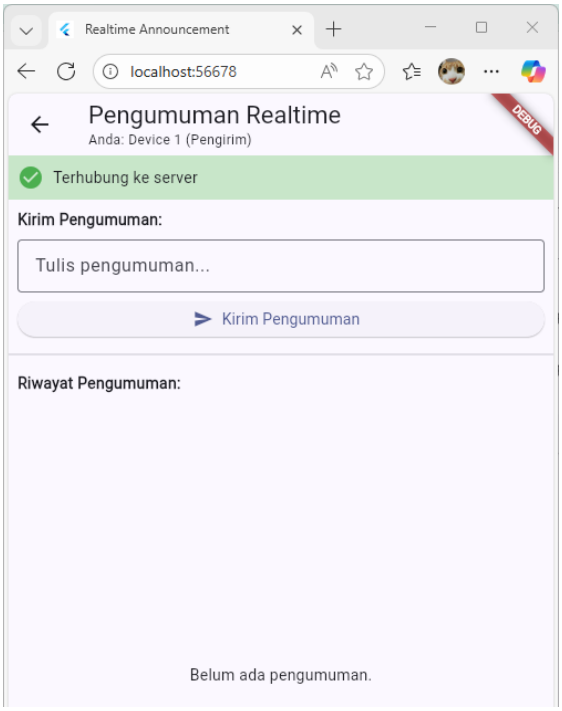
> **Realtime Announcement System** – Sistem pengumuman realtime yang memungkinkan setiap perangkat mengirim pengumuman ke perangkat lain, dan setiap penerima akan mendapatkan notifikasi lokal di HP mereka.

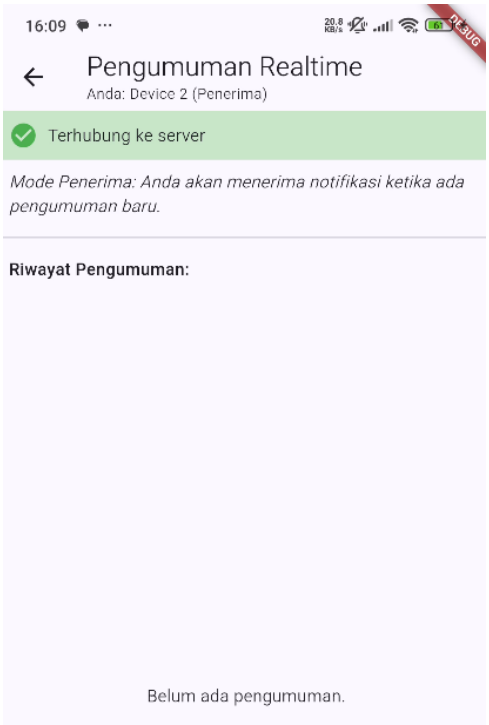
Hasil akhir aplikasi yang diharapkan:

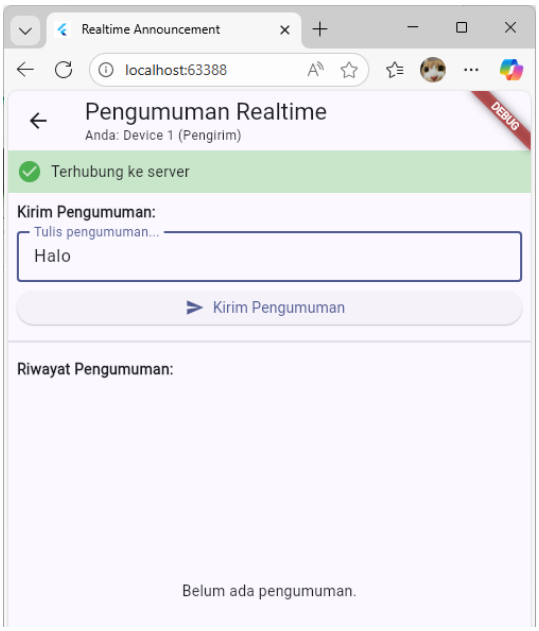


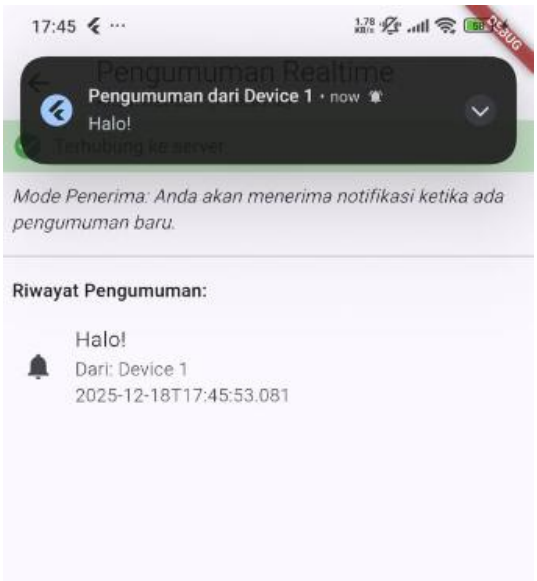



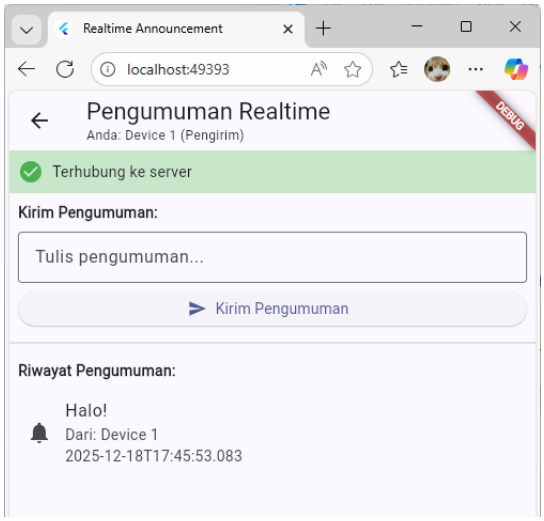



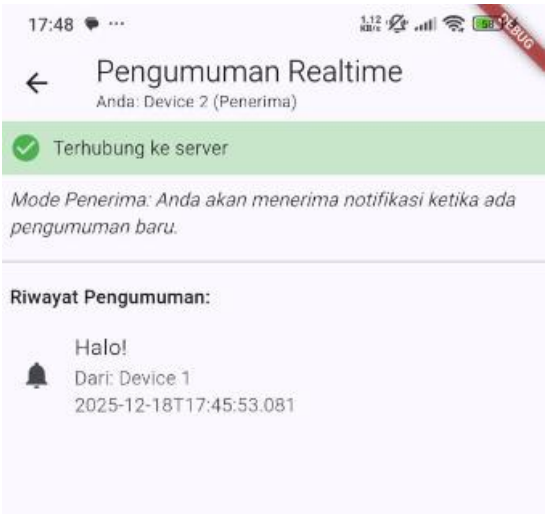

Realtime Announcement
localhost:56678
Pengumuman Realtime
Anda: Device 1 (Pengirim)
DEBUG
Terhubung ke server
Kirim Pengumuman:
Tulis pengumuman...
Kirim Pengumuman
Riwayat Pengumuman:
Belum ada pengumuman.


16:09
DEBUG
Pengumuman Realtime
Anda: Device 2 (Penerima)
Terhubung ke server
Mode Penerima: Anda akan menerima notifikasi ketika ada pengumuman baru.
Riwayat Pengumuman:
Belum ada pengumuman.


Realtime Announcement
localhost:63388
Pengumuman Realtime
Anda: Device 1 (Pengirim)
DEBUG
Terhubung ke server
Kirim Pengumuman:
Tulis pengumuman...
Halo
Kirim Pengumuman
Riwayat Pengumuman:
Belum ada pengumuman.


17:45
DEBUG
Pengumuman Realtime
Pengumuman dari Device 1 • now
Halo!
Mode Penerima: Anda akan menerima notifikasi ketika ada pengumuman baru.
Riwayat Pengumuman:
Halo!
Dari: Device 1
2025-12-18T17:45:53.081




---

# 2. Arsitektur Sistem

## 2.1. Gambaran Umum

Sistem terdiri dari dua komponen utama:

1. **Backend Server (Node.js + Socket.IO)**
    - Menangani koneksi dari semua client (aplikasi Flutter).
    - Menerima event pengumuman dari pengirim.
    - Meneruskan (broadcast) pengumuman ke semua client lain **kecuali pengirim**.
2. **Aplikasi Flutter (Client)**
    - Terhubung ke server lewat Socket.IO.
    - Dapat mengirim pengumuman (text) ke server.
    - Dapat menerima pengumuman dari server.
    - Menampilkan pengumuman yang diterima sebagai **local notification** di Android.

## 2.2. Alur Kerja Sederhana

1. Perangkat membuka aplikasi Flutter, memasukkan **nama** dan memilih **mode**:
    - Mode **"Pengirim"** → dapat mengirim pengumuman.

    - Mode **"Penerima"** → hanya menerima pengumuman.
2. Saat pengirim menekan tombol **Kirim Pengumuman**:
    - Aplikasi mengirim event `announcement:send` ke server, berisi:
        - `senderName`
        - `message`
3. Server menerima event `announcement:send`, lalu:
    - Menyimpan pengumuman ke memori (opsional).
    - Mengirim event `announcement:received` ke semua client lain **kecuali pengirim**, berisi:
        - `senderName`
        - `message`
        - `timestamp`
4. Setiap client yang menerima event `announcement:received`:
    - Menampilkan notifikasi lokal "Pengumuman baru dari {senderName}".
    - Menambahkan pengumuman ke daftar pada layar.

Dengan desain ini, **pengirim tidak akan menerima notifikasi dari pesan yang ia kirim sendiri**, karena Socket.IO mengirim broadcast ke semua client **kecuali** pengirim.

---

# 3. Prasyarat dan Persiapan Lingkungan

Sebelum mulai, pastikan hal-hal berikut sudah siap:

## 3.1. Software yang Dibutuhkan

1. **Node.js** (disarankan versi LTS terbaru)
   Unduh dari: https://nodejs.org
2. **Flutter SDK** (versi stabil terbaru)
   Panduan instalasi: https://docs.flutter.dev/get-started/install
3. **VS Code** dengan:
    - Android SDK
    - Emulator Android (opsional, bisa diganti dengan device fisik)
4. **Beberapa perangkat Android**:
    - Semua perangkat terhubung ke **jaringan Wi-Fi yang sama** dengan laptop/PC server.

## 3.2. Cek Instalasi

Buka terminal/Command Prompt, jalankan:

```
node -v
```

```
npm -v
```

```
flutter --version
```

Pastikan versi muncul dan tidak ada error.

---

# 4. Bagian 1: Membangun Backend Server (Node.js + Socket.IO)

Pada bagian ini kita akan membuat server Socket.IO sederhana yang:

- Menangani koneksi client.
- Menerima event `announcement:send`.
- Mengirim event `announcement:received` ke semua client lain (broadcast).

## 4.1. Membuat Folder Proyek Server

1. Buat folder baru, misalnya:

```
mkdir flutter-realtime-server
```

```
cd flutter-realtime-server
```

2. Inisialisasi Node.js:

```
npm init -y
```

3. Instal **Express** dan **Socket.IO**:

```
npm install express socket.io
```

## 4.2. Membuat File server.js

Di dalam folder flutter-realtime-server, buat file bernama server.js, isi dengan kode berikut:

```
const express = require('express');
const http = require('http');
const { Server } = require('socket.io');
const app = express();
const server = http.createServer(app);
const io = new Server(server, {
    cors: {
        origin: '*',
        methods: ['GET', 'POST'],
    },
});
const PORT = 3000;
const announcements = [];
io.on('connection', (socket) => {
    console.log('Client connected:', socket.id);

    // Kirim riwayat pengumuman awal
    socket.emit('announcement:history', announcements);

    // Terima event ketika ada pengumuman baru dari client
    socket.on('announcement:send', (data) => {
        console.log('New announcement from', data.senderName, ':', data.message);
        const announcement = {
            senderName: data.senderName,
            message: data.message,
            timestamp: data.timestamp,
        };
        announcements.push(announcement);

        // Broadcast ke semua client lain (kecuali pengirim)
        socket.broadcast.emit('announcement:received', announcement);

    });

    // Event ketika client terputus
    socket.on('disconnect', () => {
        console.log('Client disconnected:', socket.id);
    });
});

// Endpoint sederhana untuk cek server
app.get('/', (req, res) => {
    res.send('Realtime Announcement Server is running.');
});

// Jalankan server
server.listen(PORT, () => {
    console.log('Server listening on port', PORT);
});
```

## 4.3. Menjalankan Server

Di terminal pada folder `flutter-realtime-server`, jalankan:

```
node server.js
```

Jika berhasil, akan muncul log:

```
Server listening on port 3000
```

## 4.4. Mengetahui IP Address Server

Agar device Android dapat terhubung, Flutter perlu menggunakan **IP address lokal** dari laptop/PC tempat server dijalankan.

- Di Windows, jalankan:

```
ipconfig
```

- Di macOS/Linux, jalankan:

```
ifconfig
```

Cari IPv4 Address atau IP di jaringan Wi-Fi, misalnya: `10.0.143.104`

Nanti di Flutter kita akan menggunakan alamat:

http://10.0.143.104:3000

(sesuaikan dengan IP Anda)

---

# 5. Bagian 2: Membuat Proyek Flutter

## 5.1. Membuat Proyek Baru

Di terminal, jalankan:

```
flutter create flutter_realtime_notification
```

```
cd flutter_realtime_notification
```

Pastikan proyek berhasil dibuat.

### 5.2. Menambahkan Dependency di `pubspec.yaml`

Eksekusi perintah berikut pada terminal di proyek flutter untuk menambahkan dependency ke `pubspec.yaml` secara otomatis

# Socket.IO client for Flutter/Dart

```
flutter pub add socket_io_client
```

# Flutter Local Notifications

```
flutter pub add flutter_local_notifications
```

---

## 6. Bagian 3: Konfigurasi Notifikasi Lokal di Android

### 6.1. Mengupdate `AndroidManifest.xml`

Buka file:

`android/app/src/main/AndroidManifest.xml`

Tambahkan permission berikut di dalam tag `<manifest>` (di atas `<application>`):

```
<uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
```

Berikut potongan kodenya:

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />

    <application
        android:label="flutter_realtime_notification"
        android:name="${applicationName}"
        android:icon="@mipmap/ic_launcher">
```

> Catatan: Pada Android 13+ (API 33+), aplikasi harus meminta izin notifikasi secara runtime. Kita akan tangani ini di kode Flutter.

---

# 7. Bagian 4: Struktur Proyek Flutter & Helper Class

Berikut beberapa file yang akan dibuat:

- `lib/services/notification_service.dart` – mengelola local notification.
- `lib/services/socket_service.dart` – mengelola koneksi Socket.IO.
- `lib/pages/announcement_page.dart` – tampilan pengumuman.
- `lib/pages/login_page.dart` – tampilan login.
- `lib/my_app.dart` – pemanggil tampilan aplikasi.
- `lib/main.dart` – entry point aplikasi.

## 7.1. Membuat `notification_service.dart`

Buat folder `lib/services`, lalu buat file: `lib/services/notification_service.dart`.

Isi dengan kode berikut:

```
import 'package:flutter/material.dart';
import 'package:flutter_local_notifications/flutter_local_notifications.dart';

class NotificationService {
  NotificationService._internal();
  static final NotificationService _instance = NotificationService._internal();
  factory NotificationService() => _instance;
  final FlutterLocalNotificationsPlugin flutterLocalNotificationsPlugin =
      FlutterLocalNotificationsPlugin();
  bool _initialized = false;

  Future<void> init() async {
    if (_initialized) return;
    const AndroidInitializationSettings initializationSettingsAndroid =
        AndroidInitializationSettings('@mipmap/ic_launcher');

    const InitializationSettings initializationSettings =
        InitializationSettings(android: initializationSettingsAndroid);
```

```
    await flutterLocalNotificationsPlugin.initialize(initializationSettings);

    _initialized = true;
  }

  Future<bool> requestNotificationPermission() async {
    final android = flutterLocalNotificationsPlugin
        .resolvePlatformSpecificImplementation<
            AndroidFlutterLocalNotificationsPlugin>();

    final granted = await android?.requestNotificationsPermission() ?? false;
    debugPrint('Notification permission granted: $granted');
    return granted;
  }

  Future<void> showNotification({
    required String title,
    required String body,
  }) async {
    const AndroidNotificationDetails androidPlatformChannelSpecifics =
        AndroidNotificationDetails(
          'realtime_announcement_channel',
          'Realtime Announcement',
          channelDescription: 'Channel untuk notifikasi pengumuman realtime',
          importance: Importance.high,
          priority: Priority.high,
          playSound: true,
        );
    const NotificationDetails platformChannelSpecifics = NotificationDetails(
      android: androidPlatformChannelSpecifics,
    );

    await flutterLocalNotificationsPlugin.show(
      DateTime.now().millisecondsSinceEpoch ~/ 1000,
      title,
      body,
      platformChannelSpecifics,
    );
  }
}
```

## 7.2. Membuat socket_service.dart

Buat file: lib/services/socket_service.dart.

```
import 'package:socket_io_client/socket_io_client.dart' as io;
import 'dart:developer';
import 'dart:ui';

typedef AnnouncementHandler = void Function(Map<String, dynamic> data);

class SocketService {
  SocketService._internal();
  static final SocketService _instance = SocketService._internal();
  factory SocketService() => _instance;

  io.Socket? _socket;

  String? _serverUrl;

  AnnouncementHandler? _onAnnouncementReceived;
  VoidCallback? _onConnected;
  VoidCallback? _onDisconnected;
```

```
  bool get isConnected => _socket?.connected ?? false;

  void connect({
    required String serverUrl,
    required AnnouncementHandler onAnnouncementReceived,
    VoidCallback? onConnected,
    VoidCallback? onDisconnected,
  }) {
    connectIfNeeded(
      serverUrl: serverUrl,
      onAnnouncementReceived: onAnnouncementReceived,
      onConnected: onConnected,
      onDisconnected: onDisconnected,
    );
  }

  // Global atau session-scope.
  void connectIfNeeded({
    required String serverUrl,
    required AnnouncementHandler onAnnouncementReceived,
    VoidCallback? onConnected,
    VoidCallback? onDisconnected,
  }) {
    _serverUrl = serverUrl;
    _onAnnouncementReceived = onAnnouncementReceived;
    _onConnected = onConnected;
    _onDisconnected = onDisconnected;

    // Jika socket sudah ada dan connected:
    // - tidak membuat koneksi baru
    // - tapi panggil onConnected supaya UI halaman baru update
    if (_socket != null && _socket!.connected) {
      _onConnected?.call();
      return;
    }

    // Jika socket ada tapi tidak connected (state nyangkut), reset dulu.
    if (_socket != null) {
      try {
        _socket!.disconnect();
        _socket!.dispose();
      } catch (_) {}
      _socket = null;
    }

    _socket = io.io(
      serverUrl,
      io.OptionBuilder()
          .setTransports(['websocket'])
          .disableAutoConnect()
          .enableReconnection()
          .setReconnectionAttempts(100)
          .setReconnectionDelay(1000)
          .build(),
    );

    _bindCoreListeners();
    _bindBusinessListeners();

    _socket!.connect();
  }

  void _bindCoreListeners() {
    final s = _socket;
    if (s == null) return;

    s.off('connect');
    s.off('disconnect');
    s.off('connect_error');
```

```
    s.off('error');

    s.onConnect((_) {
      log('Socket connected: ${s.id}');
      _onConnected?.call();
    });

    s.onDisconnect((_) {
      log('Socket disconnected');
      _onDisconnected?.call();
    });

    s.onConnectError((data) {
      log('Connect error: $data');
      // Optional: anggap sebagai disconnected agar UI tidak "nyangkut hijau"
      _onDisconnected?.call();
    });

    s.onError((data) {
      log('Socket error: $data');
    });
  }

  void _bindBusinessListeners() {
    final s = _socket;
    if (s == null) return;

    s.off('announcement:received');

    s.on('announcement:received', (data) {
      if (data is Map) {
        _onAnnouncementReceived?.call(Map<String, dynamic>.from(data));
      }
    });
  }

  void sendAnnouncement({required String senderName, required String message}) {
    final s = _socket;
    if (s == null || !s.connected) {
      log('sendAnnouncement ignored: socket not connected');
      return;
    }

    final payload = {
      'senderName': senderName,
      'message': message,
      'timestamp': DateTime.now().toIso8601String(),
    };

    s.emit('announcement:send', payload);
  }

  void disconnect() {
    final s = _socket;
    if (s == null) return;

    try {
      s.disconnect();
      s.dispose();
    } catch (e) {
      log('disconnect error: $e');
    } finally {
      _socket = null;
    }
  }

  void reconnect() {
    if (_serverUrl == null || _onAnnouncementReceived == null) return;
```

```
    connectIfNeeded(
      serverUrl: _serverUrl!,
      onAnnouncementReceived: _onAnnouncementReceived!,
      onConnected: _onConnected,
      onDisconnected: _onDisconnected,
    );
  }
}
```

---

# 8. Bagian 5: Implementasi UI Flutter

Kita akan membuat UI sederhana dengan alur:

1. Halaman pertama: **Login Sederhana**
    - Input nama.
    - Pilih mode:
        - **Sebagai Pengirim**
        - **Sebagai Penerima**
    - Tombol "Masuk".
2. Halaman kedua: **Layar Pengumuman**
    - Menampilkan status koneksi ke server.
    - Menampilkan list pengumuman yang diterima.
    - Jika mode Pengirim:
        - Terdapat TextField + tombol "Kirim Pengumuman".
    - Jika mode Penerima:
        - Hanya list + info "Anda sebagai penerima".

## 8.1. Membuat tampilan UI

lib/pages/announcement_page.dart:

```
import 'package:flutter/material.dart';
import 'package:flutter_realtime_notification/services/notification_service.dart';
import 'package:flutter_realtime_notification/services/socket_service.dart';
import 'login_page.dart';

class AnnouncementPage extends StatefulWidget {
  final String userName;
  final UserRole role;

  const AnnouncementPage({
    super.key,
    required this.userName,
    required this.role,
  });

  @override
```

```
  State<AnnouncementPage> createState() => _AnnouncementPageState();
}

class _AnnouncementPageState extends State<AnnouncementPage> {
  final SocketService _socketService = SocketService();
  final TextEditingController _messageController = TextEditingController();
  final List<Map<String, dynamic>> _announcements = [];

  bool _isConnected = false;
  static const String _serverUrl =
      'http://10.0.143.104:3000'; // sesuaikan IP Anda

  @override
  void initState() {
    super.initState();
    _connectSocket();
  }

  void _connectSocket() {

    setState(() => _isConnected = _socketService.isConnected);

    _socketService.connect(
      serverUrl: _serverUrl,
      onAnnouncementReceived: (data) async {
        setState(() {
          _announcements.insert(0, data);
        });

        final senderName = data['senderName'] ?? 'Unknown';
        final message = data['message'] ?? '';

        await NotificationService().showNotification(
          title: 'Pengumuman dari $senderName',
          body: message,
        );
      },
      onConnected: () {
        setState(() => _isConnected = true);
      },
      onDisconnected: () {
        setState(() => _isConnected = false);
      },
    );
  }

  void _sendAnnouncement() {
    final message = _messageController.text.trim();
    if (message.isEmpty) {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Pesan pengumuman tidak boleh kosong')),
      );
      return;
    }

    _socketService.sendAnnouncement(
      senderName: widget.userName,
      message: message,
    );

    setState(() {
      _announcements.insert(0, {
        'senderName': widget.userName,
        'message': message,
        'timestamp': DateTime.now().toIso8601String(),
      });
    });

    _messageController.clear();
```

```
  }

  @override
  void dispose() {
    _messageController.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    final isSender = widget.role == UserRole.sender;

    return Scaffold(
      appBar: AppBar(
        title: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const Text('Pengumuman Realtime'),
            Text(
              'Anda: ${widget.userName} (${isSender ? 'Pengirim' : 'Penerima'})',
              style: Theme.of(context).textTheme.bodySmall,
            ),
          ],
        ),
      ),
      body: Column(
        children: [
          Container(
            color: _isConnected ? Colors.green.shade100 : Colors.red.shade100,
            padding: const EdgeInsets.all(8.0),
            width: double.infinity,
            child: Row(
              children: [
                Icon(
                  _isConnected ? Icons.check_circle : Icons.error,
                  color: _isConnected ? Colors.green : Colors.red,
                ),
                const SizedBox(width: 8),
                Text(
                  _isConnected
                      ? 'Terhubung ke server'
                      : 'Tidak terhubung ke server',
                ),
              ],
            ),
          ),
          if (isSender)
            Padding(
              padding: const EdgeInsets.all(8.0),
              child: Column(
                children: [
                  const Align(
                    alignment: Alignment.centerLeft,
                    child: Text(
                      'Kirim Pengumuman:',
                      style: TextStyle(fontWeight: FontWeight.bold),
                    ),
                  ),
                  const SizedBox(height: 8),
                  TextField(
                    controller: _messageController,
                    decoration: const InputDecoration(
                      labelText: 'Tulis pengumuman...',
                      border: OutlineInputBorder(),
                    ),
                    minLines: 1,
                    maxLines: 3,
                  ),
                  const SizedBox(height: 8),
```

```
                    SizedBox(
                      width: double.infinity,
                      child: ElevatedButton.icon(
                        onPressed: _isConnected ? _sendAnnouncement : null,
                        icon: const Icon(Icons.send),
                        label: const Text('Kirim Pengumuman'),
                      ),
                    ),
                  ],
                ),
              )
            else
              const Padding(
                padding: EdgeInsets.all(8.0),
                child: Text(
                  'Mode Penerima: Anda akan menerima notifikasi ketika ada pengumuman
baru.',
                  style: TextStyle(fontStyle: FontStyle.italic),
                ),
              ),
            const Divider(),
            const Padding(
              padding: EdgeInsets.all(8.0),
              child: Align(
                alignment: Alignment.centerLeft,
                child: Text(
                  'Riwayat Pengumuman:',
                  style: TextStyle(fontWeight: FontWeight.bold),
                ),
              ),
            ),
            Expanded(
              child: _announcements.isEmpty
                  ? const Center(child: Text('Belum ada pengumuman.'))
                  : ListView.builder(
                      itemCount: _announcements.length,
                      itemBuilder: (context, index) {
                        final item = _announcements[index];
                        final senderName = item['senderName'] ?? '';
                        final message = item['message'] ?? '';
                        final timestamp = item['timestamp'] ?? '';

                        return ListTile(
                          leading: const Icon(Icons.notifications),
                          title: Text(message),
                          subtitle: Text(
                            'Dari: $senderName\n$timestamp',
                            maxLines: 2,
                            overflow: TextOverflow.ellipsis,
                          ),
                        );
                      },
                    ),
            ),
          ],
        ),
      );
    }
  }
```

lib/pages/login_page.dart:

```
import 'package:flutter/material.dart';
import 'announcement_page.dart';

enum UserRole { sender, receiver }

class LoginPage extends StatefulWidget {
  const LoginPage({super.key});

  @override
  State<LoginPage> createState() => _LoginPageState();
}

class _LoginPageState extends State<LoginPage> {
  final TextEditingController _nameController = TextEditingController();
  UserRole _role = UserRole.sender;

  @override
  void dispose() {
    _nameController.dispose();
    super.dispose();
  }

  void _goToAnnouncementPage() {
    final name = _nameController.text.trim();
    if (name.isEmpty) {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Nama tidak boleh kosong')),
      );
      return;
    }

    Navigator.of(context).push(
      MaterialPageRoute(
        builder: (_) => AnnouncementPage(userName: name, role: _role),
      ),
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Login Sederhana')),
      body: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Column(
          children: [
            TextField(
              controller: _nameController,
              decoration: const InputDecoration(
                labelText: 'Nama Anda',
                border: OutlineInputBorder(),
              ),
            ),
            const SizedBox(height: 16),
            const Align(
              alignment: Alignment.centerLeft,
              child: Text(
                'Pilih Mode:',
                style: TextStyle(fontWeight: FontWeight.bold),
              ),
            ),

            RadioGroup<UserRole>(
              groupValue: _role,
              onChanged: (UserRole? value) {
                if (value == null) return;
                setState(() => _role = value);
```

```
              },
              child: const Column(
                children: [
                  RadioListTile<UserRole>(
                    title: Text('Sebagai Pengirim (Sender)'),
                    value: UserRole.sender,
                  ),
                  RadioListTile<UserRole>(
                    title: Text('Sebagai Penerima (Receiver)'),
                    value: UserRole.receiver,
                  ),
                ],
              ),
            ),

            const SizedBox(height: 16),
            SizedBox(
              width: double.infinity,
              child: ElevatedButton(
                onPressed: _goToAnnouncementPage,
                child: const Text('Masuk'),
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

lib/my_app.dart:

```
import 'package:flutter/material.dart';
import 'pages/login_page.dart';

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Realtime Announcement',
      theme: ThemeData(
        useMaterial3: true,
        colorSchemeSeed: Colors.indigo,
      ),
      home: const LoginPage(),
    );
  }
}
```

lib/main.dart:

```
import 'package:flutter/material.dart';
import 'services/notification_service.dart';
import 'my_app.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await NotificationService().init();
  await NotificationService().requestNotificationPermission();
  runApp(const MyApp());
}
```

**Catatan penting:** Ganti `_serverUrl` di `AnnouncementPage` dengan IP server Anda sendiri, misalnya:

```
static const String _serverUrl = 'http:// 10.0.143.104:3000';
```

---

# 9. Bagian 6: Menjalankan & Menguji Aplikasi

## 9.1. Menjalankan Server

Pastikan server sudah berjalan:

```
cd flutter-realtime-server
```

```
node server.js
```

Biarkan terminal ini tetap terbuka.

## 9.2. Menjalankan Aplikasi Flutter di Beberapa Perangkat

1. Hubungkan 2–3 perangkat Android ke laptop (atau gunakan emulator).
2. Pastikan semua perangkat terhubung ke **Wi-Fi yang sama** dengan server.
3. Jalankan perintah:

```
flutter devices
```

Pastikan semua device terdeteksi.

4. Jalankan aplikasi ke salah satu device:

```
flutter run
```

Atau dari VS Code, pilih target device dan klik Run.

5. Ulangi untuk device lain (bisa menggunakan `flutter run -d <device_id>` untuk menjalankan ke device yang berbeda).

## 9.3. Skenario Pengujian

### Test 1 – Dua Device (A dan B)

1. **Device A**:
    - Buka aplikasi.
    - Masukkan nama, misalnya: `Andi`.
    - Pilih mode **Sebagai Pengirim (Sender)**.
    - Tap **Masuk**.
2. **Device B**:
    - Buka aplikasi.
    - Masukkan nama, misalnya: `Budi`.
    - Pilih mode **Sebagai Penerima (Receiver)**.
    - Tap **Masuk**.
3. **Device A (Andi)**:
    - Ketik pesan: `Kuliah hari ini dimundurkan 15 menit`.
    - Tap **Kirim Pengumuman**.
4. Amati:
    - **Device A**: tidak muncul notifikasi, hanya menampilkan pengumuman di list UI.
    - **Device B**: muncul **notifikasi lokal** dengan judul `Pengumuman dari Andi` dan isi pesan yang dikirim.

### Test 2 – Tiga Device (A, B, C)

Tambahkan **Device C** sebagai penerima atau pengirim.

- Jika **C sebagai penerima**:
    - Set seperti Budi (Receiver).
    - Saat Andi mengirim pengumuman:
        - Budi dan C menerima notifikasi.
        - Andi tidak.
- Jika **B sebagai pengirim**:
    - Ubah mode Budi menjadi **Sender** saat login.
    - Saat Budi mengirim pengumuman:
        - Andi dan C menerima notifikasi.
        - Budi tidak.

Dengan demikian, perilaku "pengirim tidak menerima notifikasi" tercapai secara alami melalui mekanisme broadcast Socket.IO yang mengecualikan pengirim.

---

# 10. Bagian 7: Troubleshooting & Catatan Penting

## 10.1. Aplikasi Tidak Terhubung ke Server

- Pastikan:
  - Server Node.js sedang berjalan (lihat terminal: `Server listening on port 3000`).
  - IP server di `_serverUrl` sudah diisi dengan benar (bukan `localhost`, gunakan IPv4 LAN).
  - Device dan laptop berada di jaringan Wi-Fi yang sama.

## 10.2. Notifikasi Tidak Muncul di Android

- Cek:
  - Permission POST_NOTIFICATIONS sudah ditambahkan di `AndroidManifest.xml`.
  - Pada Android 13+, user harus mengizinkan notifikasi ketika prompt muncul.
  - Cobalah reinstall aplikasi dan pastikan izin notifikasi diaktifkan.

## 10.3. Pesan Tidak Sampai ke Device Lain

- Cek log di terminal server:
  - Apakah event `announcement:send` tercetak?
- Cek log di aplikasi Flutter:
  - Tambahkan `print()`/`log()` ke listener `onAnnouncementReceived`.

## 10.4. Perilaku di Background

- Aplikasi ini menggunakan **local notification**, sehingga:
  - Saat aplikasi berada di **foreground** atau **background ringan**, notifikasi biasanya tetap muncul.
  - Jika aplikasi di-**force stop** dari App Info, notifikasi tidak akan muncul lagi sampai aplikasi dibuka kembali (ini **batasan normal** Android untuk local notification yang dipicu dari dalam aplikasi).

---

# 11. Pengembangan Lanjutan

Praktikum ini dapat dikembangkan lebih lanjut agar:

1. Menambahkan field **"Judul Pengumuman"** terpisah dari isi pesan.
2. Menambahkan filter pengumuman berdasarkan **kategori**.
3. Menyimpan pengumuman ke **database** (misalnya MySQL).
4. Menambahkan fitur **"baca/tidak terbaca"** pada list pengumuman.

---

# 12. Ringkasan

Dalam praktikum ini, kita telah:

- Membuat **server Node.js + Socket.IO** sederhana untuk broadcast pengumuman.
- Membuat aplikasi Flutter yang:
  - Terhubung ke server dengan **Socket.IO**.
  - Mengirim dan menerima event pengumuman.
  - Memicu **local notification** ketika menerima pengumuman dari perangkat lain.
- Menguji aplikasi pada beberapa perangkat Android dan mengamati:
  - Pengirim tidak menerima notifikasi dari pesan yang ia kirim.
  - Penerima mendapat notifikasi lokal secara realtime.

Struktur dan kompleksitas contoh ini sengaja dibuat **sesederhana mungkin** namun tetap relevan dengan kebutuhan nyata, sehingga mudah dipahami dan dapat dijadikan dasar untuk proyek yang lebih kompleks di kemudian hari.